

EXPERIMENT NO. 08**AIM**

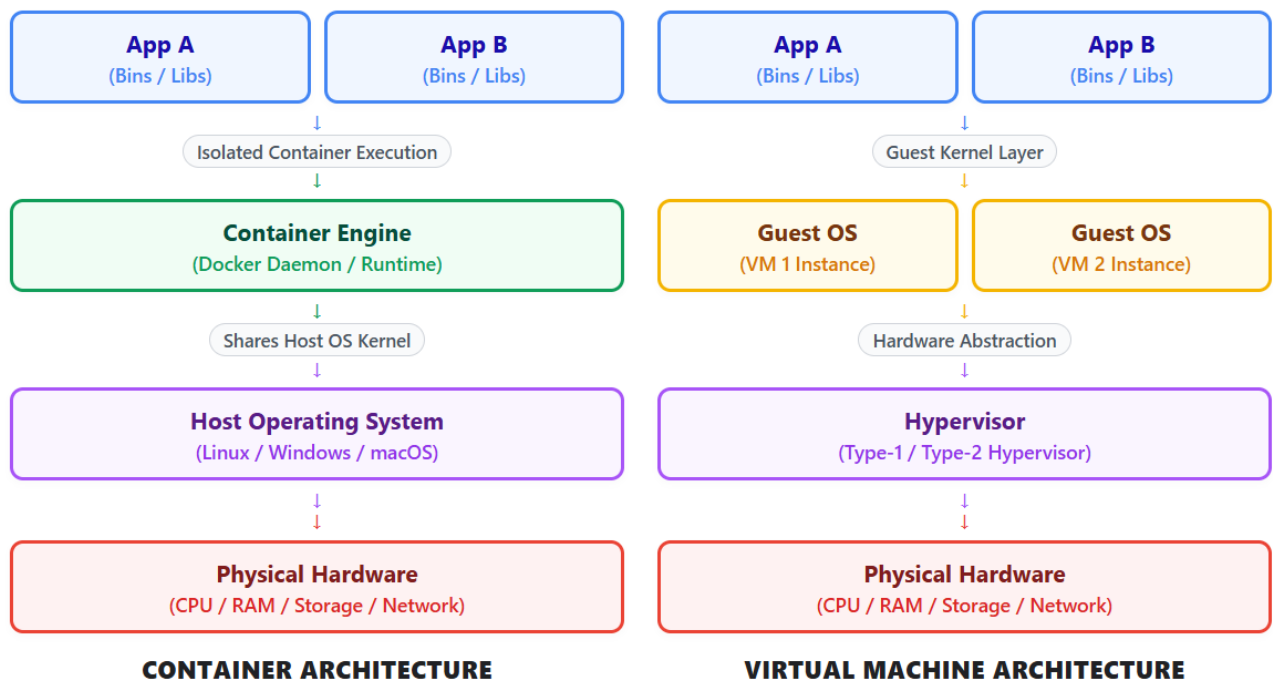
Deploy the application using Docker and GitHub.

LEARNING OUTCOMES

Deploy full-stack application on cloud platform and present capstone.

THEORY**Fundamentals of Containerization & Docker**

- **Containerization:** A lightweight OS-level virtualization method that packages application source code together with its runtime dependencies, libraries, system tools, and configuration files into an isolated execution unit called a container.
- **Containers vs. Virtual Machines (VMs):**
 - **Virtual Machines** abstract hardware, requiring a full guest operating system on top of a hypervisor, which consumes significant memory, storage, and boot time.
 - **Docker Containers** share the host system's OS kernel, isolating process spaces and filesystems. This makes containers lightweight (megabytes vs. gigabytes), fast to start (milliseconds vs. minutes), and resource efficient.



- **Core Docker Artifacts:**
 - **Dockerfile:** A declarative text blueprint containing step-by-step instructions to assemble a Docker image.
 - **Docker Image:** An immutable, read-only template built in stacked filesystem layers.
 - **Docker Container:** A runnable, stateful instance of a Docker image operating in an isolated user space.

Multi-Stage Docker Builds

- A **Docker** feature that uses multiple FROM statements in a single Dockerfile. Each FROM instruction begins a new build stage using a distinct base image.
- Separation of Build-time vs. Runtime Environments:
 - **Build Stage:** Utilizes heavyweight SDK base images containing compilers, build tools, and package managers (e.g., maven:3.9 or node:18) to compile source code into execution binaries (.jar or optimized React bundle).
 - **Runtime Stage:** Copies only the final compiled artifacts into a clean, minimal runtime image (e.g., eclipse-temurin:17-jre-alpine or nginx:alpine).

Multi-Container Orchestration with Docker Compose

- A tool for defining and running multi-container Docker applications through a single declarative YAML configuration file (docker-compose.yml).
- **Isolated Virtual Networking:**
 - Docker Compose automatically initializes an isolated bridge network (fullstack-net).
 - **Service DNS Name Resolution:** Containers attached to the same custom bridge network resolve other containers using their defined service names (e.g., mysqlldb or backend) as domain aliases rather than ephemeral container IP addresses
- **Container Lifecycle & Dependency Management:**
 - **depends_on:** Defines the start and stop order of dependent services.
 - **healthcheck:** Executes status commands inside a container (e.g., mysqladmin ping) to verify that an internal service engine (like MySQL) is fully ready to accept socket connections before downstream containers (like Spring Boot) launch.
- **Data Persistence via Docker Volumes:**
 - Containers are ephemeral by default; data written inside a container is lost when it stops.
 - **Named Volumes:** Managed persistent storage spaces created on the host file system and mounted into specific container paths (e.g., /var/lib/mysql), ensuring database records persist across container rebuilds or host reboots.

CI/CD Workflows with GitHub Actions

- **Continuous Integration (CI):** The practice of automatically triggering build, syntax-check, and automated test pipelines every time code changes are committed and pushed to a central repository (e.g., GitHub).
- **Continuous Delivery (CD):** Automatically preparing, packaging, and staging verified build artifacts (such as Docker images) into container registries (e.g., Docker Hub, GitHub Packages) for deployment to production environments.

- GitHub Actions Workflow Architecture:
 - **Events (on):** Triggers that launch workflows, such as push or pull_request to the main branch.
 - **Jobs:** A set of sequential steps executing on a isolated virtual machine runner (e.g., ubuntu-latest).
 - **Steps & Actions:** Tasks executing specific shell commands (docker build) or reusing pre-built community actions (e.g., actions/checkout or docker/setup-buildx-action).

CODE

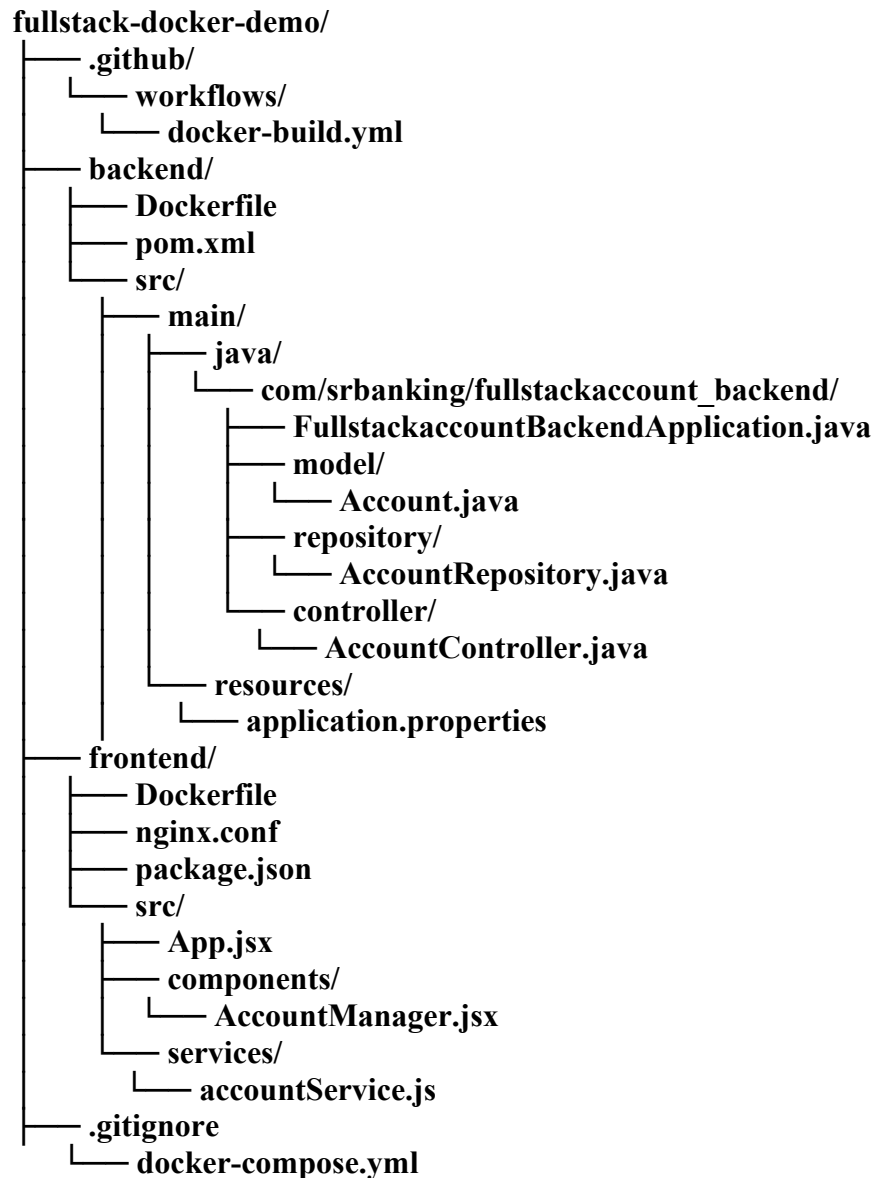
Prerequisites and Setup

Before starting, ensure the following software is installed on your computer:

- **Docker Desktop:** Download and install Docker Desktop. Open Docker Desktop after installation and make sure it shows "**Engine Running**".
- **Git:** Download and install Git.
- **GitHub Account:** Create a free account at github.com.

Step - 1: Set up the project directory

- Create a main project folder on your computer named fullstack-docker-demo.
- Open **VS Code**, go to **File > Open Folder...**, and select fullstack-docker-demo.
- Create the following folder structure inside VS Code



File - 1: Spring Boot Database Connection
(backend/src/main/resources/application.properties)

In Docker, containers communicate using service names instead of localhost. Set the database URL host to mysqldb:

```
server.port=8082
```

```
spring.application.name=account-demo
```

```
spring.datasource.url=jdbc:mysql://mysqldb:3306/banking_db?useSSL=false&serverTimezone=UTC&allowPublicKeyRetrieval=true
```

```
spring.datasource.username=root
```

```
spring.datasource.password=rootpass
```

```
spring.datasource.driver-class-name=com.mysql.cj.jdbc.Driver
```

```
spring.jpa.hibernate.ddl-auto=update
spring.jpa.show-sql=true
spring.jpa.properties.hibernate.dialect=org.hibernate.dialect.MySQLDialect
```

File - 2: Backend Container Instructions (backend/Dockerfile)

Create a file named Dockerfile (no extension) inside the backend folder:

```
FROM maven:3.9-eclipse-temurin-17 AS builder
WORKDIR /app
COPY pom.xml .
RUN mvn dependency:go-offline
COPY src ./src
RUN mvn clean package -DskipTests

FROM eclipse-temurin:17-jre-alpine
WORKDIR /app
COPY --from=builder /app/target/*.jar app.jar
EXPOSE 8082
ENTRYPOINT ["java", "-jar", "app.jar"]
```

File - 3: Frontend Nginx Proxy Configuration (frontend/nginx.conf)

Create a file named nginx.conf inside the frontend folder to handle routing and API requests:

```
server {
    listen 80;

    location / {
        root /usr/share/nginx/html;
        index index.html index.htm;
        try_files $uri $uri/ /index.html;
    }

    location /api/ {
        proxy_pass http://backend:8082/api/;
        proxy_set_header Host $host;
        proxy_set_header X-Real-IP $remote_addr;
        proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
    }
}
```

File - 4: Frontend Container Instructions (frontend/Dockerfile)

Create a file named Dockerfile inside the frontend folder:

```
FROM node:18-alpine AS builder
WORKDIR /app
COPY package*.json ./
RUN npm install
COPY . .
RUN npm run build

FROM nginx:alpine
COPY --from=builder /app/dist /usr/share/nginx/html
COPY nginx.conf /etc/nginx/conf.d/default.conf
EXPOSE 80
CMD ["nginx", "-g", "daemon off;"]
```

File - 5: Multi-Container Orchestration (docker-compose.yml)

Create docker-compose.yml in the root project folder:

```
version: '3.8'

services:
  mysqldb:
    image: mysql:8.0
    container_name: mysqldb
    restart: always
    environment:
      MYSQL_ROOT_PASSWORD: rootpass
      MYSQL_DATABASE: banking_db
    ports:
      - "3306:3306"
    volumes:
      - db_data:/var/lib/mysql
    healthcheck:
      test: ["CMD", "mysqladmin", "ping", "-h", "localhost", "-u", "root", "-prootpass"]
      interval: 10s
      timeout: 5s
      retries: 5
    networks:
      - fullstack-net

  backend:
    build:
      context: ./backend
      dockerfile: Dockerfile
    container_name: backend-service
    restart: always
```

```
ports:
  - "8082:8082"
depends_on:
  mysql:
    condition: service_healthy
networks:
  - fullstack-net

frontend:
  build:
    context: ./frontend
    dockerfile: Dockerfile
  container_name: frontend-service
  restart: always
  ports:
    - "80:80"
  depends_on:
    - backend
  networks:
    - fullstack-net

volumes:
  db_data:

networks:
  fullstack-net:
    driver: bridge
```

File - 6: GitHub Actions Workflow (.github/workflows/docker-build.yml)

Create docker-build.yml inside .github/workflows/:

name: CI/CD Docker Build Workflow

```
on:
  push:
    branches: [ "main" ]
  pull_request:
    branches: [ "main" ]

jobs:
  build-and-test:
    runs-on: ubuntu-latest

    steps:
      - name: Checkout Source Code
        uses: actions/checkout@v3
```

- name: Set up Docker Buildx
uses: docker/setup-buildx-action@v2
- name: Build Backend Docker Image
run: |
docker build -t backend-service:latest ./backend
- name: Build Frontend Docker Image
run: |
docker build -t frontend-service:latest ./frontend
- name: Validate Docker Compose Configuration
run: |
docker compose config

Step - 3: Run the application locally with docker compose

- Open the VS Code integrated terminal (Ctrl + ~ or **Terminal > New Terminal**).
- Ensure you are in the root directory (fullstack-docker-demo).
- Run the following command to build images and launch all containers in detached (background) mode:
docker compose up --build -d

What happens during this step:

- Docker downloads the mysql:8.0, maven:3.9, and node:18 images.
- Docker compiles Java source code into .jar artifacts and builds React assets.
- Docker starts MySQL, waits for it to become healthy, launches Spring Boot, and starts Nginx.
- Verify running containers:
docker compose ps

All three services (mysqldb, backend-service, frontend-service) should show Up.

- Test in Browser: Open your browser and go to <http://localhost>
- Add a new account record. The data persists automatically into MySQL inside the Docker volume.

Step - 4: Push to Github and Trigger Automated CI

- Go to github.com, click the + icon at top right, and select **New repository**.
- Name the repository fullstack-docker-demo, select Public, and click Create repository.
- Copy the repository URL provided by GitHub.
- Open your VS Code terminal and run the following commands sequentially:
git init
git add .
git commit -m "Initial commit - Fullstack Docker and CI Setup"
git branch -M main
git remote add origin <https://github.com/<YOUR-GITHUB-USERNAME>/fullstack-docker-demo.git>
git push -u origin main

(Replace <YOUR-GITHUB-USERNAME> with your actual GitHub username).

- Verify GitHub Actions Pipeline:
 - Open your GitHub repository in your browser.
 - Click the **Actions** tab.
 - You will see the **CI/CD Docker Build Workflow** running automatically. Click on it to inspect real-time containers build logs.

Step - 5: Helpful Docker Commands

- View live logs of all services:
docker compose logs -f
- View logs for a specific service (e.g., backend):
docker compose logs -f backend
- Stop all running containers:
docker compose down
- Stop containers and delete database volume (fresh reset):
docker compose down -v

CONCLUSION

.....

.....

.....

PRACTICAL RELATED QUESTIONS

1. What is the primary benefit of using multi-stage builds in Dockerfiles for React and Spring Boot applications?
2. How does Docker Compose enable communication between frontend, backend, and database containers without hardcoding IP addresses?
3. Why is depends_on alone insufficient for controlling startup order between MySQL and Spring Boot, and how does healthcheck fix this?

.....

.....

.....

.....

.....

.....

.....

This image shows a full page of white paper with horizontal dotted lines, typical of primary school writing paper. The lines are evenly spaced and run across the width of the page. There are no margins, text, or other markings on the paper.

